

UNIT- III

CONVOLUTIONAL NEURAL NETWORKS

18TE7F3 – Deep Learning and Artificial
Intelligence

7th sem - 2022-2023 (Odd semester)

Unit – III Syllabus

Convolutional Neural Networks (CNN): Introduction to CNN, Components of CNN: Convolution layer, Pooling layer, Flattening layer, Fully connected layer, ReLU layer, Exponential linear unit, Properties of CNN, Architectures of CNN-LeNet, AlexNet, ZF-Net, VGGNet, GoogLeNet, ResNet, DenseNet, Applications of CNN.

Recurrent Neural Networks (RNN): Introduction to RNN, Training of RNN, Back propagation through time (BPTT) illustration, RNN Topology, Challenges with Vanishing gradients, Bidirectional RNNs, Long Short term Memory (LSTM), Gated Recurrent unit (GRU), Deep Recurrent Neural Networks, Applications of RNN.

Auto Encoders: Introduction to auto encoders Features of auto encoders, Types of Auto encoders, Vanilla auto encoder, Multilayer auto encoder, stacked auto encoder, Deep Auto encoder, denoising autoencoder, convolutional autoencoder, Regularization in autoencoder (regularized autoencoder). Applications of Auto encoders.

Learning Objectives

- Understand Basics of CNN and various components in its architecture.
- Analyze the effects of different activation function of a CNN unit.
- Understand CNN properties, architectural variants and their applications.

Convolutional Neural Network (CNN) Overview

- ✓ **Convolutional Neural Network (CNN)** is a class of **deep, feed-forward neural networks**.
- ✓ Specially designed for **visual imagery** with **minimal preprocessing**.
- ✓ Learns **features automatically**, from **basic patterns** → **complex structures**

Key Concepts

- ✓ **Neurons + Learnable Parameters**
 - ✓ CNNs contain neurons with trainable weights & biases (like standard neural networks)

Convolutional Neural Network (CNN) Overview

✓ **Mathematical Convolution:**

- ✓ Core operation where a **filter (kernel)** slides over the image to extract features

Network Architecture:

- ✓ Input Layer → **Multiple Hidden Layers (Convolution, Activation, Pooling, etc.)** → Output Layer

- The system learns to do **feature extraction**, and the core concept of CNN is that it uses **convolution of images and filters to generate invariant features**, which are passed on to the next layer.
- Inspired by the **human visual cortex** – neurons activate when seeing specific edges and patterns.
- In traditional machine learning, *humans must manually design the features* that the model will use.
- Example in image classification:
 - You may manually compute **edges, corners, textures, color histograms, shapes**, etc.
 - Then you feed these handcrafted numbers into **ML algorithms like SVM, KNN, or Random Forest**.

Problem:

This requires domain expertise, time, and may **miss important hidden patterns**.

- In CNN (Convolutional Neural Network): The model **learns these features by itself directly from images**, without humans explicitly telling it what to look for.
- CNN figures out: **where edges exist, what shapes form a face, object, digit, etc. what texture and patterns are useful for classification**
- In object detection, both **humans and machine learning models** create bounding boxes, but at different stages.
- **Human annotators** create the initial ground-truth bounding boxes for training data,
- while the **machine learning model** predicts bounding boxes during its actual operation to identify and localize objects

Manual Vs Automatic

Manual Feature Design

Human selects edges, corners etc.

Requires expertise

May miss patterns

Less accurate for images

CNN Automatic Feature Learning

CNN learns them on its own

No manual intervention

Learns best possible features from data

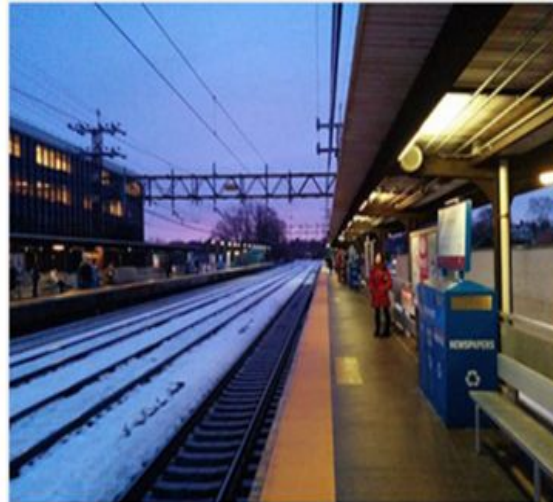
Very powerful for image tasks

Convolutional Neural Networks Intuition

- ConvNets have been successful in identifying **faces**, **objects** and **traffic signs** apart from powering vision in **robots** and **self driving cars**



night bridge city suspension bridge
river



train subway railroad railway station
transportation



competition tennis athlete stadium ball
many spectators

In Figure above, a ConvNet is able to recognize scenes and the system is able to suggest relevant tags such as ‘bridge’, ‘railway’ and ‘tennis’

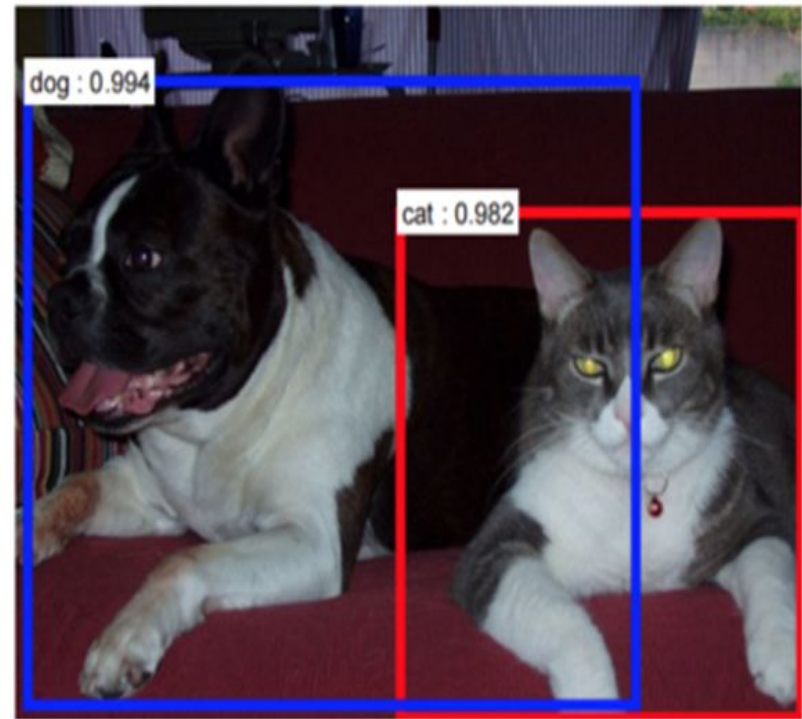
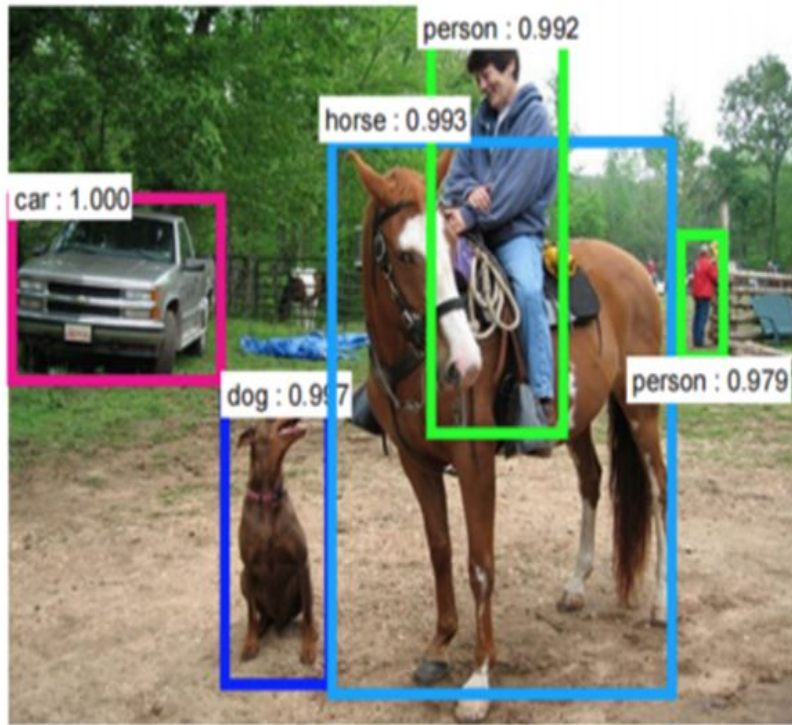


Figure shows an example of ConvNets being used for recognizing everyday object, humans and animals. ConvNets, therefore, are an important tool for most machine learning practitioners today.

Terminology

Term

Meaning

Weight Matrix

Trainable numeric values inside filters that get updated during learning

Filter / Kernel

Small matrix (e.g., 3×3 , 5×5) used to detect patterns like edges, curves, textures

Features

Distinct patterns in the image (edges $\rightarrow$ corners $\rightarrow$ shapes $\rightarrow$ objects)

Feature Map

Output generated after applying a filter on the input image

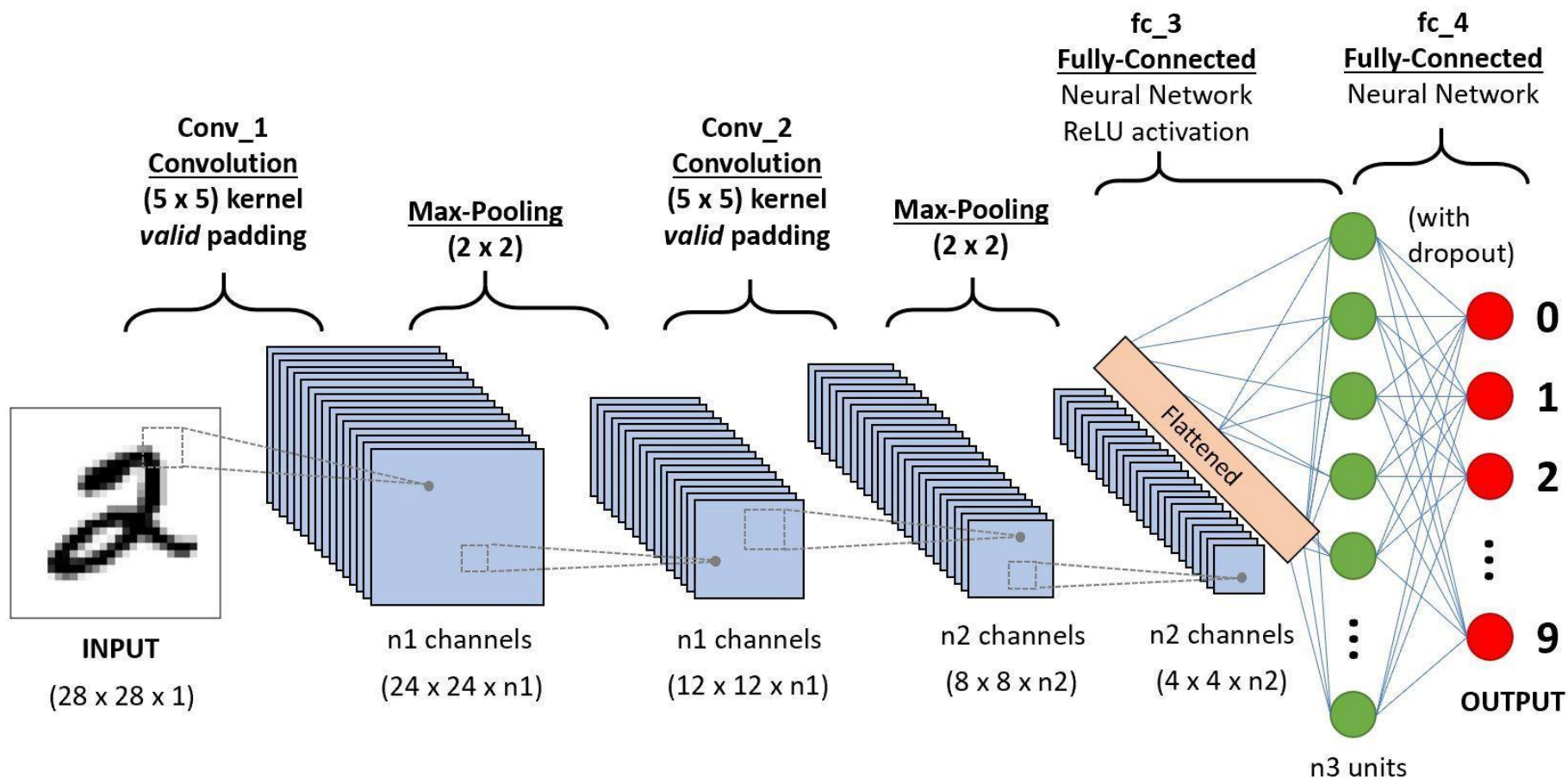
CNN

Applications

- Self-Driving Cars
- Robotics & Automation
- Face Recognition
- Medical Image Diagnosis
- Object Detection, Image Classification

- YOLO, Faster R-CNN, Mask R-CNN, U-Net, SegNet, ResNet,
- EfficientNet
- VGG-Face, FaceNet, MTCNN, OpenFace, MobileFaceNet
- U-Net, ResNet, DenseNet, VGG, EfficientNet
- AlexNet, VGG16, ResNet, YOLO, SSD, RetinaNet, Mask-RCNN/CNN

Structure of CNN



Structure of CNN

Term

Meaning (how to tell in class)

Input size ($H_{in} \times W_{in}$)

“This is the height and width of the image we give to CNN.”
Example: 28×28 image

Kernel size ($K \times K$)

“This is the small box (filter) that slides over the image to detect features.” Common sizes: 3×3 or 5×5

Stride (S)

“How many steps the filter moves at a time.” Stride 1: move 1 pixel at a time (slow, detailed) Stride 2: jump 2 pixels (faster, smaller output)

Padding (P)

“Adding extra border pixels around the image so that corner data is not lost and size can be controlled.” Zero padding means we add 0s around the border

Output size ($H_o \times W_o$)

“The final size of the feature map after the filter finishes sliding over the image.”

Structure of CNN

Given:

Input = 7×7 image

Kernel = 3×3

Stride = 1

Padding = 0

$$Output = \frac{7 - 3 + 2(0)}{1} + 1 = 5$$

So the feature map will be 5×5

- Input = 7
- Kernel = 3
- Stride = 1
- Padding = 1

$$Output = \frac{7 - 3 + 2(1)}{1} + 1 = 7$$

Structure of CNN

Pool1 (Max-Pooling 2×2, stride 2)

Pooling *reduces height and width*, formula:

$$H_{out} = \frac{H_{in}}{S}$$

Parameter	Value
(H_{in})	24
Stride	2

$$H_{out}=24/2=12$$

Output after Pool1: **$12 \times 12 \times n_1$**

Structure of CNN

Conv2 (5×5 kernel, stride 1, VALID padding)

Apply convolution formula again:

$$H_{out} = \frac{(12 - 5 + 2(0))}{1} + 1 = 8$$

Output after Conv2: **8 × 8 × n₂**....

(n₂ = number of filters in Conv2, so depth increases)

Pool2 (Max-Pooling 2×2, stride 2)

$$H_{out} = \frac{8}{2} = 4$$

Output after Pool2: **4 × 4 × n₂**

Flatten

Flattening means converting the 3D tensor (height × width × depth) into a 1D vector:

$$4 \times 4 \times n_2 = 16n_2 \text{ units}$$

Flatten result: **16 · n₂ neurons** → fed into fully connected (dense) layers

Components of CNN

- **Key points about Convolution layers and Filters**
- **The depth of a filter in a CNN must match the depth of the input image.** The number of color channels in the filter must remain the same as the input image.
- **Different Conv2D filters are created for each of the three channels** for a color image.
- **Filters for each layer are randomly initialized based on either Normal or Gaussian distribution.**
- **Early layers have fewer filters; deeper layers have more filters**
- Early layers extract *simple features*.
- Deep layers extract *complex features*.

Components of CNN

Layer Level	What filters learn	Typical # of filters
Shallow layers (Conv1, Conv2)	Edges, curves, simple patterns	8, 16, 32
Middle layers	Shapes, textures	64, 128
Deep layers	High-level features (eyes, objects, digits)	256, 512

More filters → more computation

- Deeper layers with many filters learn better features, **but require more computation and memory.**

Consider the convolutional neural network defined by the layers in the left column below. Fill in the shape of the output volume and the number of parameters at each layer. You can write the shapes in the NumPy format (e.g. (128,128,3)).

Notation:

- CONV5-N denotes a convolutional layer with N filters with height and width equal to 5. Padding is 2, and stride is 1.
- POOL2 denotes a 2x2 max-pooling layer with stride of 2 and 0 padding.
- FC-N denotes a fully connected layer with N neurons.

Layer	Activation Volume Dimensions	Number of parameters
Input	32*32*1	0
CONV5-10		
POOL2		
CONV5-10		
POOL2		
FC10		

Convolution Layer

- Filters or kernels are the base units in this system.
- These layers consist of a series of learnable filters, meaning the CNN adjusts their values during training.
- Convolution is done by computing the dot product between the entries of the filter and the volume.
- Let us understand the process of convolution with an example.

(a)

0	1	1	0	0
0	0	1	0	0
0	1	1	0	0
0	0	1	0	0
0	1	1	0	0

(a) Input Matrix

(b)

0	1	1
0	0	1
0	1	1

(b) filter or kernel

Feature Map Creation

(a)

0x0	1x1	1x1	0	0
0x0	0x0	1x1	0	0
0x0	1x1	1x1	0	0
0	0	1	0	0
0	1	1	0	0

(b)

5		

(a) First subset of the input feature; (b) first value of the feature map.

(a)

0	1x0	1x1	0x1	0
0	0x0	1x0	0x1	0
0	1x0	1x1	0x1	0
0	0	1	0	0
0	1	1	0	0

(b)

5	2	

(a) Second subset of the input feature; (b) second value of the feature map.

Parameters

Term	Meaning
Input size ($H \times W$)	“This is the height and width of the image we give to CNN.” Example: 28×28 image
Kernel size ($K \times K$)	“This is the small box (filter) that slides over the image to detect features.” Common sizes: 3×3 or 5×5
Stride (S)	“How many steps the filter moves at a time.” Stride 1: move 1 pixel at a time (slow, detailed) Stride 2: jump 2 pixels (faster, smaller output)
Padding (P)	“Adding extra border pixels around the image so that corner data is not lost and size can be controlled.” Zero padding means we add 0s around the border
Output size ($H_o \times W_o$)	“The final size of the feature map after the filter finishes sliding over the image.”

Feature Map

- The size of the feature map (convolved feature) is controlled by three parameters (to be decided before the convolution)
-
- **Depth:** no. of filters, depth of feature map n
 - **Stride:** filter matrix pixel slid over the input matrix
 - **Zero Padding:** Adjusting the input size as per the requirement by adding zeros to the input matrix.

Padding

- **Padding in Convolutional Neural Networks (CNNs)** refers to adding extra layers of pixels around the edges of an input image or feature map.
- This technique is used to control the spatial dimensions of the output after a convolution operation.

Why Use Padding?

1. Preserve Spatial Dimensions:

1. Without padding, applying filters reduces the dimensions of the input feature map, especially near the edges.

2. Prevent Information Loss:

1. Filters inherently struggle to process the edges of an image because there are fewer surrounding pixels.

3. Allow Deeper Networks:

1. Maintaining dimensions across layers (using padding) helps design deeper networks without quickly shrinking the feature maps.

Padding

Zero padding (most common): add rows/columns of zeros around the input before convolving.

Why use it

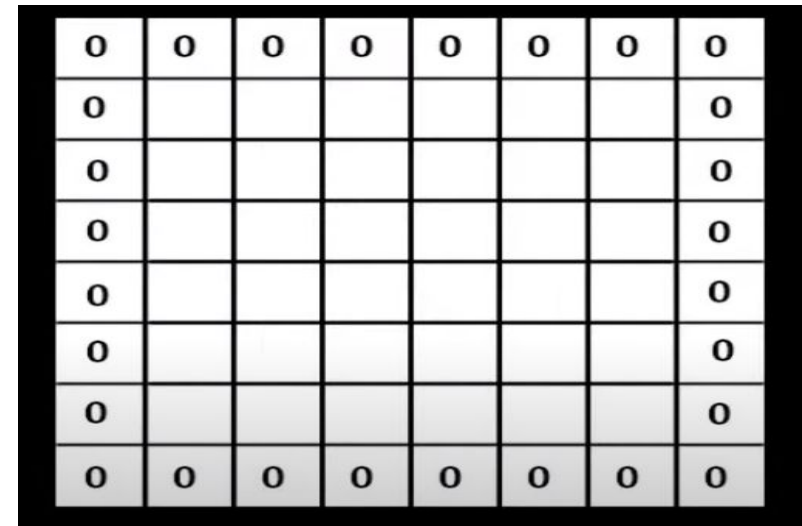
- Simple and cheap to compute.
- Preserves more spatial information near borders than “valid” (no padding).

Valid padding (“no padding”)

Definition: do not add any padding; convolution is only applied where the filter entirely overlaps the input.

What it looks like:

With $K = 3$ on a 5×5 input, the filter can only be placed in positions that keep it fully inside the image $\rightarrow$ output shrinks.



0	0	0	0	0	0	0	0
0							0
0							0
0							0
0							0
0							0
0							0
0	0	0	0	0	0	0	0

Padding

1. Same Padding:

1. Adds enough padding so the output dimensions are the same as the input dimensions.
2. Ensures consistent size across layers.

Step 1: Original 5×5 Image

1	2	3	4	5
6	7	8	9	1
1	3	5	7	9
2	4	6	8	1
9	8	7	6	5

0	0	0	0	0	0	0
0	1	2	3	4	5	0
0	6	7	8	9	1	0
0	1	3	5	7	9	0
0	2	4	6	8	1	0
0	9	8	7	6	5	0
0	0	0	0	0	0	0

Zero Padding = The method of adding zeros around the image.

Same Padding = A goal/strategy that uses padding (usually zeros) so that the **output size stays the same as the input size**.

Step 3: Perform convolution (using 3×3 kernel)

Since we added padding, the filter can slide over **every original pixel**, including edges.

Output size formula:

$$H_{out} = \frac{H_{in} + 2P - K}{S} + 1$$

Substitute:

$$H_{out} = \frac{5 + 2(1) - 3}{1} + 1 = 5$$

Output size stays **5 × 5**

That is why it's called **SAME padding** → Same input size = Same output size

Representation of Stride

0	1	1	0	0
0	0	1	0	0
0	1	1	0	0
0	0	1	0	0
0	1	1	0	0

0	1	1	0	0
0	0	1	0	0
0	1	1	0	0
0	0	1	0	0
0	1	1	0	0

0	1	1	0	0
0	0	1	0	0
0	1	1	0	0
0	0	1	0	0
0	1	1	0	0

Stride : 1
Move 1 column to right at
each step

0	1	1	0	0
0	0	1	0	0
0	1	1	0	0
0	0	1	0	0
0	1	1	0	0

0	1	1	0	0
0	0	1	0	0
0	1	1	0	0
0	0	1	0	0
0	1	1	0	0

Stride : 2
Move 2 columns to right
at each step

Example-1: Numerical Problems

- Given the input matrix and the kernel, perform convolution with stride being 1 and 2.

1	0	1	1	0
0	0	0	1	1
1	0	0	0	1
0	1	1	1	0
1	1	0	1	0

Input matrix

1	0	0
0	0	1
1	1	0

Kernel

- Perform convolution with stride 1

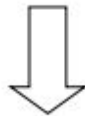
Step 1:

1	0	1	1	0
0	0	0	1	1
1	0	0	0	1
0	1	1	1	0
1	1	0	1	0

Input

1	0	0
0	0	1
1	1	0

Kernel



2		

Feature Map

Step 2:

1	0	1	1	0
0	0	0	1	1
1	0	0	0	1
0	1	1	1	0
1	1	0	1	0

Input

1	0	0
0	0	1
1	1	0

Kernel



2	1	

Feature Map

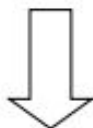
Step 3:

1	0	1	1	0
0	0	0	1	1
1	0	0	0	1
0	1	1	1	0
1	1	0	1	0

Input

1	0	0
0	0	1
1	1	0

Kernel



2	1	2

Feature Map

Step 4:

1	0	1	1	0
0	0	0	1	1
1	0	0	0	1
0	1	1	1	0
1	1	0	1	0

Input

1	0	0
0	0	1
1	1	0

Kernel



2	1	2
1		

Feature Map

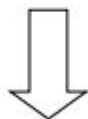
Step 5:

1	0	1	1	0
0	0	0	1	1
1	0	0	0	1
0	1	1	1	0
1	1	0	1	0

Input

1	0	0
0	0	1
1	1	0

Kernel



2	1	2
1	2	

Feature Map

Step 6:

1	0	1	1	0
0	0	0	1	1
1	0	0	0	1
0	1	1	1	0
1	1	0	1	0

Input

1	0	0
0	0	1
1	1	0

Kernel



2	1	2
1	2	3

Feature Map

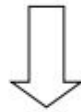
Step 7:

1	0	1	1	0
0	0	0	1	1
1	0	0	0	1
0	1	1	1	0
1	1	0	1	0

Input

1	0	0
0	0	1
1	1	0

Kernel



2	1	2
1	2	3
4		

Feature Map

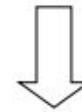
Step 8:

1	0	1	1	0
0	0	0	1	1
1	0	0	0	1
0	1	1	1	0
1	1	0	1	0

Input

1	0	0
0	0	1
1	1	0

Kernel



2	1	2
1	2	3
4	2	

Feature Map

Step 9:

1	0	1	1	0
0	0	0	1	1
1	0	0	0	1
0	1	1	1	0
1	1	0	1	0

Input

1	0	0
0	0	1
1	1	0

Kernel



2	1	2
1	2	3
4	2	1

Feature Map after applying stride1

- Perform convolution with stride 2

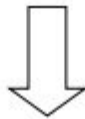
Step 1:

1	0	1	1	0
0	0	0	1	1
1	0	0	0	1
0	1	1	1	0
1	1	0	1	0

Input

1	0	0
0	0	1
1	1	0

Kernel



2	

Feature Map

Step 2:

1	0	1	1	0
0	0	0	1	1
1	0	0	0	1
0	1	1	1	0
1	1	0	1	0

Input

1	0	0
0	0	1
1	1	0

Kernel



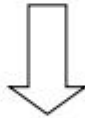
2	2

Feature Map

Step 3:

1	0	1	1	0
0	0	0	1	1
1	0	0	0	1
0	1	1	1	0
1	1	0	1	0

Input



2	2
1	

Feature Map

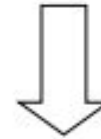
1	0	0
0	0	1
1	1	0

Kernel

Step 4:

1	0	1	1	0
0	0	0	1	1
1	0	0	0	1
0	1	1	1	0
1	1	0	1	0

Input



2	2
1	3

Feature Map

1	0	0
0	0	1
1	1	0

Kernel

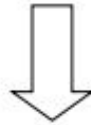
Step 5:

1	0	1	1	0
0	0	0	1	1
1	0	0	0	1
0	1	1	1	0
1	1	0	1	0

Input

1	0	0
0	0	1
1	1	0

Kernel



2	2
1	3
4	

Feature Map

Step 6:

1	0	1	1	0
0	0	0	1	1
1	0	0	0	1
0	1	1	1	0
1	1	0	1	0

Input

1	0	0
0	0	1
1	1	0

Kernel



2	2
1	3
4	1

Feature Map for Stride 2

Example -2:

INPUT

1	1	1	0	0
0	1	1	1	0
0	0	1	1	1
0	0	1	1	0
0	1	1	0	0

1	1	1	0	0
0	1	1	1	0
0	0	1	1	1
0	0	1	1	0
0	1	1	0	0

1	1	1	0	0
0	1	1	1	0
0	0	1	1	1
0	0	1	1	0
0	1	1	0	0

1	1	1	0	0
0	1	1	1	0
0	0	1	1	1
0	0	1	1	0
0	1	1	0	0

FILTER

1	0	1
0	1	0
1	0	1

1	0	1
0	1	0
1	0	1

1	0	1
0	1	0
1	0	1

1	0	1
0	1	0
1	0	1

CONVOLVED FEATURE

4		

4		
2		

4	3	
2		

4	3	4
2	4	3
2	3	4

Filter or Kernels

Sharpening filter $\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$

Laplacian filter $\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$

Sobel x-axis filter $\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$

Pooling or Down sampling Layer

- Pooling Layer function is to progressively *reduce the spatial size* of the representation to reduce the number of parameters and computation in the network.
- Periodically insert pooling layer in between successive convolution layer
- Resize operation
- Hence to also control overfitting.
- Spatial pooling is called *subsampling or downsampling*.
- It reduces the dimensionality of each map but retains the important information.

Types Of Spatial Pooling

- **Max-Pooling:**

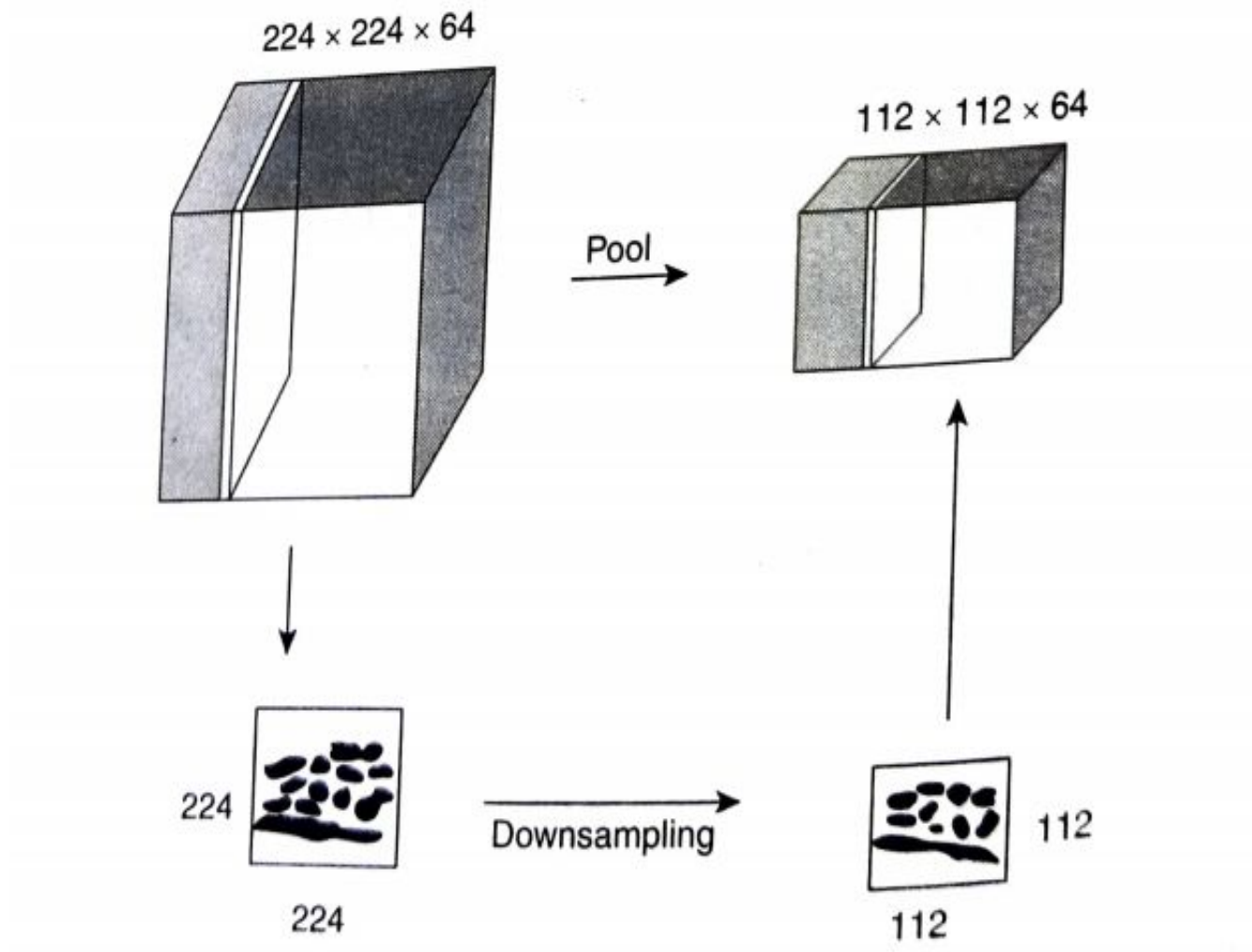
Max-pooling takes the *largest element* from the rectified feature map. Ex. Sharp edges, curves

- **Average Pooling:**

Average pooling works well for straight lines and smaller curves (smooth fashion), but cannot detect extreme features like sharp edges.

- **Sum Pooling:**

The sum of all elements in the feature map is called sum pooling. Decreases the dimension of input data.



Flattening Layer

- Fully connected layers cannot process multidimensional data
- So reduced down to single dimension (flattened)

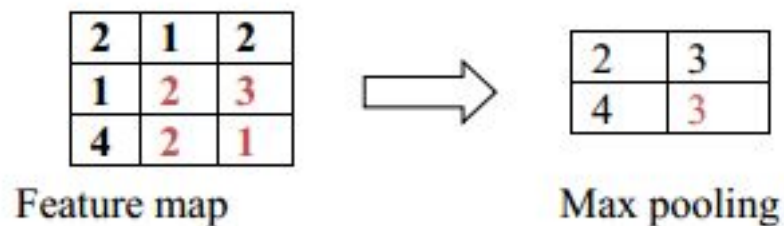
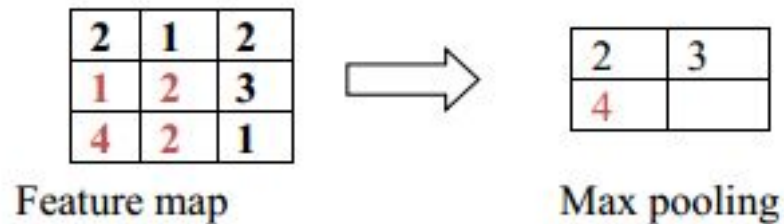
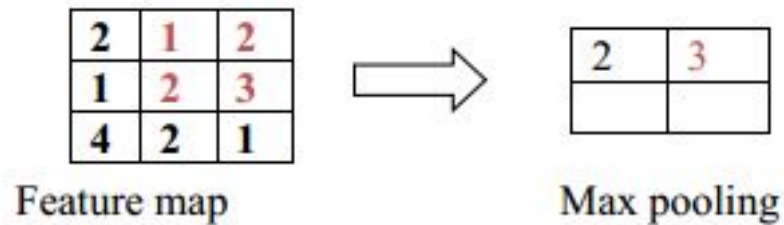
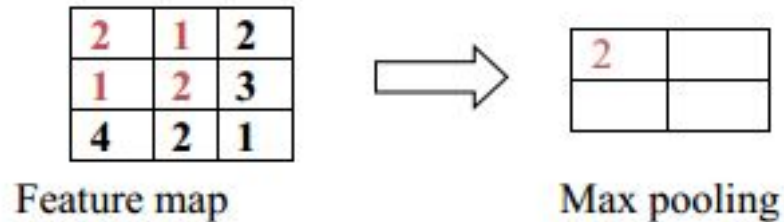
Fully connected layer

- FC layer is a traditional MLP.
- Every neuron in the previous layer is connected to every neuron on the next layer.
- Classification based on training dataset.
- Output probabilities

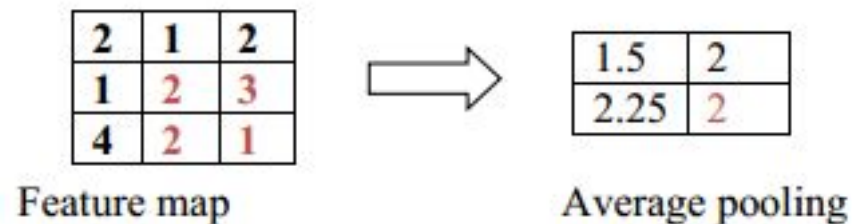
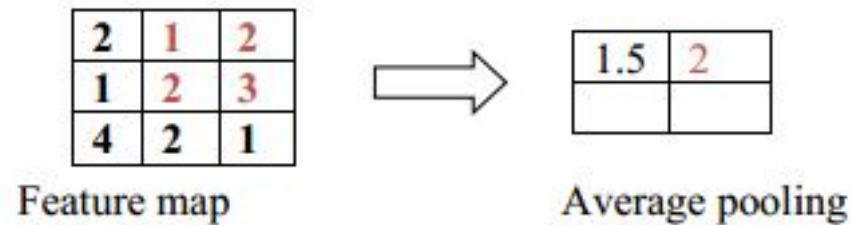
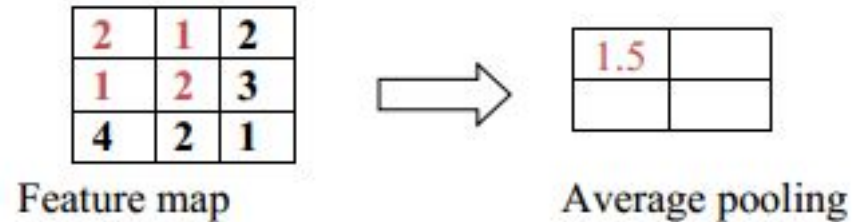
Question: Apply Max-pooling, Average-pooling, and Sum-pooling to the results of convolutions
(Example-1)

Feature map after applying stride1.

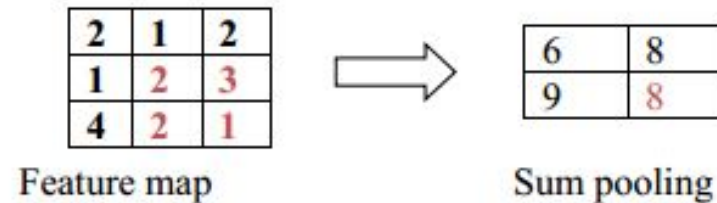
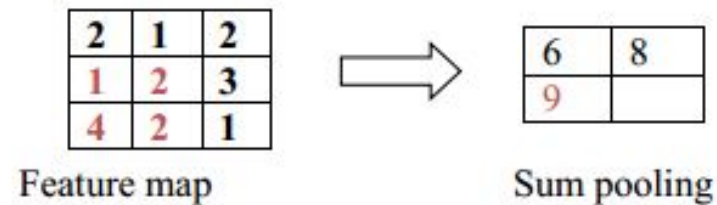
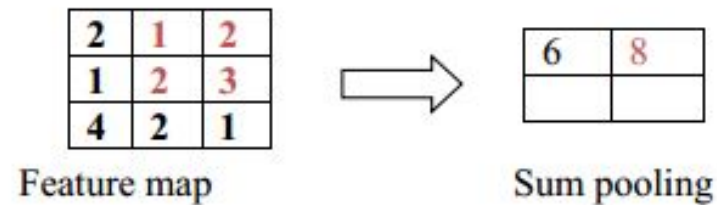
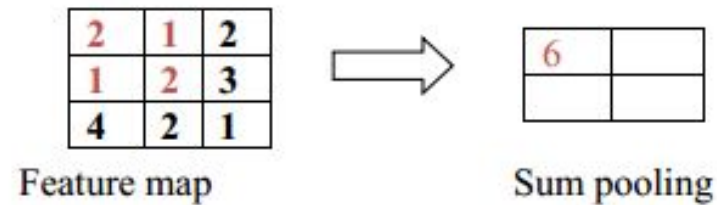
- Apply max pooling



- Apply average pooling

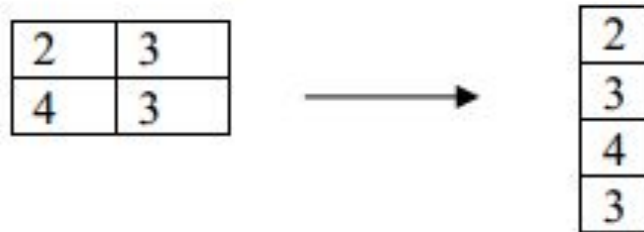


- Apply Sum pooling

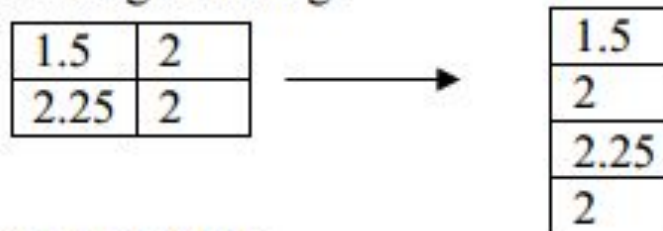


- flattened version of the pooled feature maps

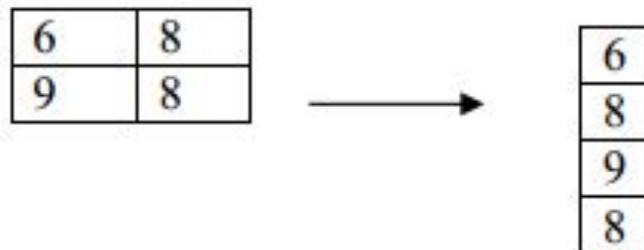
Max Pooling:



Average Pooling:



Sum Pooling:



Rectified Linear Unit (ReLU)

- Activation layer is applied immediately after each convolution layer to introduce **non linearity** in the convolution layers.
- It also alleviates the vanishing gradient problem.
- The ReLU layer applies the below function to all of the values in the input volume.

$$f(x) = \max(0, x)$$

- There are other non-linear functions such as tanh or sigmoid, but performance-wise ReLU is better than the other two.

Variants of ReLU

- **Leaky ReLU**

Leaky ReLU helps increase the range of the ReLU function. Leaky ReLU has a small slope for negative values.

- **Randomized ReLU**

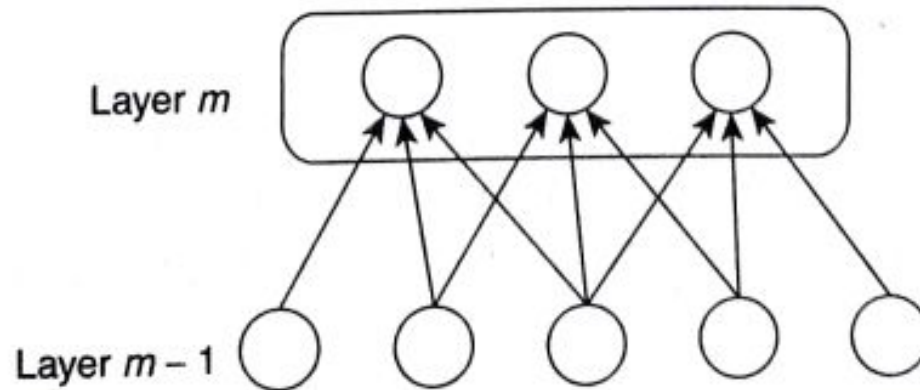
Usually, the activation value a is 0.01 or so. When a is not 0.01 then it is called randomized ReLU.

Properties of CNN

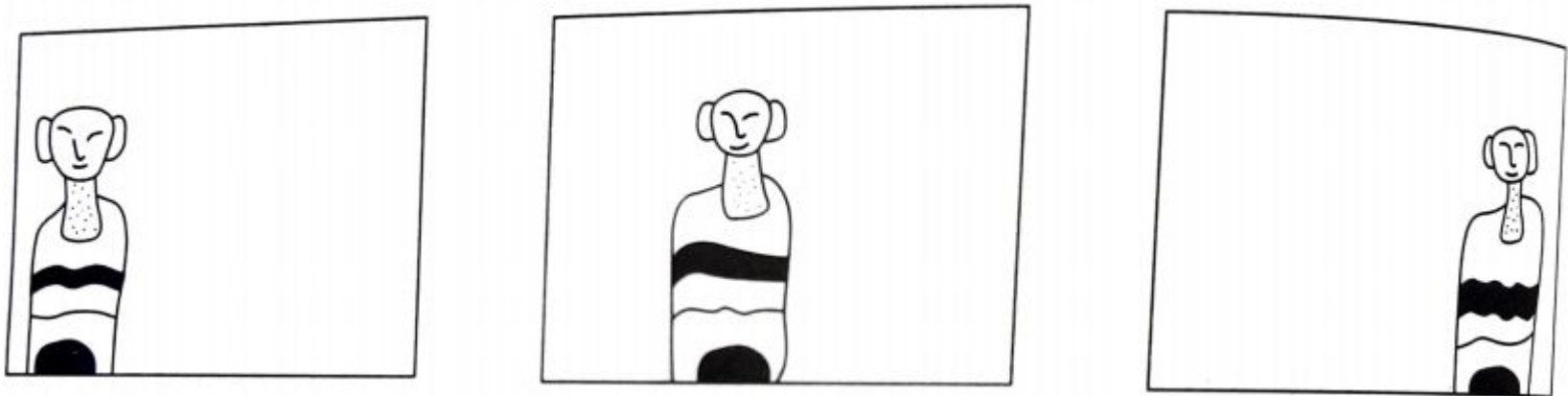
- **Weight Sharing:** weight sharing is advantageous in some of the following ways:
 - It reduces the number of parameters
 - It avoids overfitting
 - Model converges faster
- **Translation Invariance:**

This property allows CNN to recognize the identified object even when it is moved to any different place.

- Neurons sharing weights



- Translation Invariant



Applications of CNN

- Multistage architecture with convolution, pooling and fully connected layers. Integral part of computer vision

Applications

- Object detection
- Face recognition
- Scene labelling
- Character or handwriting recognition.

Object Detection

- Object detection is a technology that deals with detecting real-world objects from a given scene.
- Instance of a particular class of object
- Deep CNN: Complex vision problems

